

Universidade de Aveiro

Departamento de Engenharia Mecânica

Vaso Inteligente – SmartPot

Projeto Final de TAC

Gonçalo Cristóvão — 109041

Resumo

Este relatório apresenta o desenvolvimento do SmartPot, um sistema IoT para monitorização e rega automática de plantas. O projeto utiliza um ESP32, sensores resistivos e uma bomba peristáltica controlada por histerese. Foram realizados ensaios de caracterização hidráulica e geométrica para otimizar o volume de rega. O sistema integra-se com a Google Firebase para armazenamento de dados e com uma aplicação móvel para controlo remoto [1].

Conteúdo

1	Introdução	3
1.1	Objetivos do Projeto	3
2	Arquitetura do Sistema	4
2.1	Diagrama de Fluxo e Lógica de Operação	4
3	Hardware e Caracterização Experimental	7
3.1	Componentes Utilizados	7
3.1.1	Microcontrolador ESP32	7
3.1.2	Sensor de Humidade do Solo (YL-69) e Calibração	7
3.1.3	Limitações e Fenómenos de Oxidação	8
3.2	Caracterização Hidráulica da Bomba	8
3.3	Dimensionamento Geométrico do Vaso	9
3.4	Montagem do Circuito e Diagrama de Ligações	9
4	Programação e Firmware	11
4.1	Lógica de Controlo e Necessidades Hídricas	11
4.1.1	Implementação do Algoritmo Adaptativo	11
4.2	Comunicação I2C	12
4.3	Sincronização Temporal e Gestão de Dados Cloud	13
4.4	Lógica de Segurança e Tratamento de Dados	13
5	Aplicação Móvel e Cloud	15
5.1	Desenvolvimento da Aplicação Móvel	15
5.2	Google Firebase (Realtime Database)	15
5.3	Lógica de Programação (Thunkable)	16
5.4	Análise da Lógica de Programação (Thunkable)	17
5.5	Interface e Fluxo do Utilizador	17
6	Análise Económica	19
7	Conclusão	20
7.1	Síntese do Trabalho e Cumprimento dos Objetivos	20
7.2	Perspetivas de Evolução e Trabalhos Futuros	20

Capítulo 1

Introdução

No contexto atual de crescente urbanização e digitalização, a integração de sistemas físicos com a rede (Internet das Coisas - IoT) tem permitido otimizar tarefas quotidianas. O cuidado com a vegetação doméstica, muitas vezes negligenciado devido à falta de tempo ou conhecimento técnico sobre as necessidades hídricas das plantas, apresenta-se como uma oportunidade para a aplicação de sistemas de controlo automático.

O projeto **SmartPot** surge como uma solução de engenharia para este problema, focando-se no desenvolvimento de um dispositivo capaz de monitorizar e intervir de forma autónoma no ecossistema de um vaso.

1.1 Objetivos do Projeto

Este trabalho tem como objetivo principal o desenvolvimento de um protótipo funcional que cumpra os seguintes requisitos:

- **Automação:** Garantir a rega autónoma baseada na humidade real do solo, utilizando lógica de histerese;
- **Conetividade:** Sincronizar dados de sensores com uma base de dados na nuvem (*Google Firebase*) via protocolo Wi-Fi;
- **Interface Humano-Máquina:** Disponibilizar ao utilizador uma aplicação móvel para monitorização remota e visualização de históricos;
- **Eficiência:** Desenvolver um sistema de baixo custo e alta fiabilidade através da calibração precisa dos componentes analógicos.

Capítulo 2

Arquitetura do Sistema

A arquitetura do sistema *SmartPot* foi desenhada para garantir uma resposta em tempo real às variações ambientais, seguindo um modelo de ciclo fechado: Aquisição → Processamento → Atuação → Visualização.

O sistema divide-se em dois grandes domínios complementares:

- **Nível Físico (Hardware e Eletrónica):** Responsável pela interação direta com o meio biológico. Este domínio engloba a leitura analógica da humidade do solo, o processamento local no ESP32 e a atuação mecânica da bomba peristáltica via relé.
- **Nível Lógico (Software e Nuvem):** Responsável pela gestão inteligente de dados e interface com o utilizador. Este domínio inclui a sincronização temporal via NTP, o armazenamento na *Google Firebase* e a interface móvel desenvolvida em *Thunkable*.

2.1 Diagrama de Fluxo e Lógica de Operação

O funcionamento lógico do firmware, desenvolvido em ambiente Arduino IDE, está representado no fluxograma da Figura 2.1. Esta lógica assegura que o microcontrolador não apenas execute tarefas sequenciais, mas responda a eventos e estados de segurança.

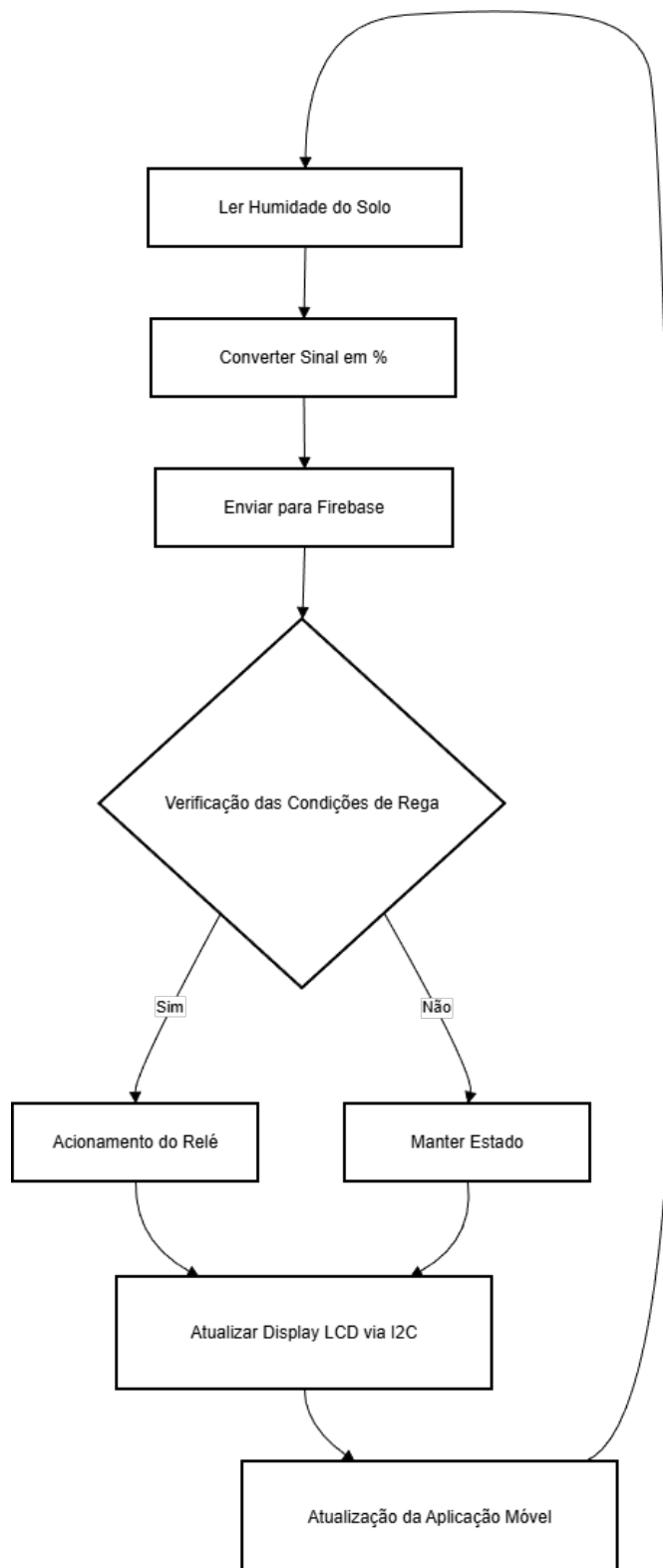


Figura 2.1: Fluxograma do algoritmo de controlo e lógica de decisão.

A lógica de operação detalhada no fluxograma pode ser segmentada em três pilares fundamentais:

- **Inicialização e Conetividade:** Ao ser alimentado, o ESP32 inicia os protocolos de comunicação Wi-Fi e sincroniza o relógio interno via NTP. Esta etapa é crítica

para garantir que os dados enviados para o Firebase tenham o carimbo temporal (*timestamp*) correto para o histórico semanal.

- **Controle por Histerese:** Para evitar o desgaste prematuro da bomba e do relé (fenômeno de *chattering*), o sistema utiliza dois limiares. A rega só é ativada no Limiar Crítico (30%) e apenas cessa quando o solo atinge o Limiar de Paragem (60%), garantindo uma hidratação profunda e estável.
- **Gestão de Segurança do Reservatório:** O sistema monitoriza o uso da bomba através de um contador de regas. Caso o limite de segurança seja atingido (prevendo tanque vazio), o algoritmo interrompe a atuação para proteger a bomba peristáltica de trabalhar em seco, emitindo um alerta visual e remoto.

Capítulo 3

Hardware e Caracterização Experimental

3.1 Componentes Utilizados

3.1.1 Microcontrolador ESP32

O ESP32 usado possui conectividade a Wi-Fi nativa e pelo seu conversor analógico-digital (ADC) de 12 bits, que permite uma resolução de 0 a 4095, essencial para a precisão na leitura dos sensores [2].

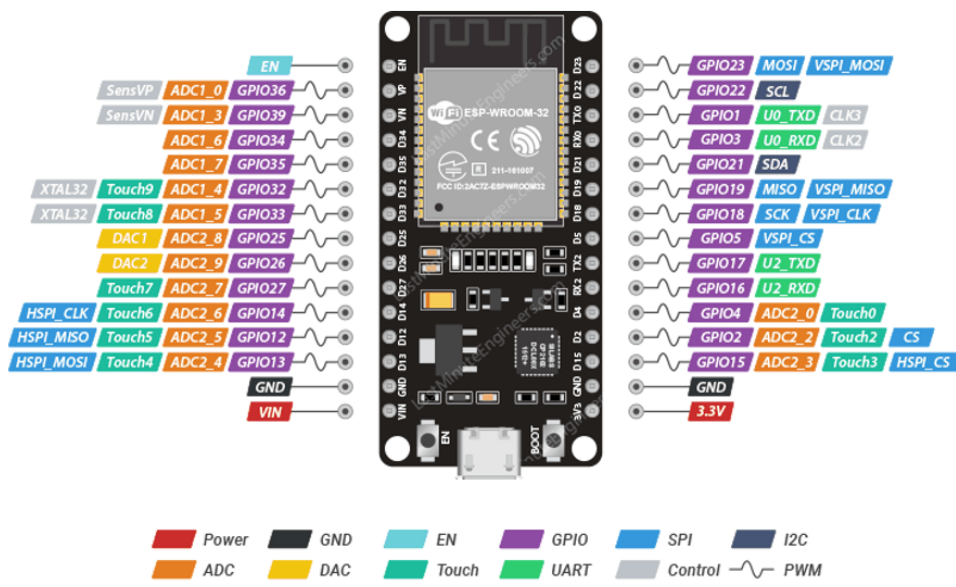


Figura 3.1: Esquema de Periféricos ESP32

3.1.2 Sensor de Humidade do Solo (YL-69) e Calibração

Utilizou-se o sensor resistivo YL-69 para monitorizar o teor de água no substrato [3]. Este sensor opera através da medição da resistência elétrica entre dois elétrodos. Para garantir

a fiabilidade do sistema, realizou-se uma calibração experimental para mapear os valores brutos do ADC em percentagem de humidade (Tabela 3.1).

Tabela 3.1: Mapeamento entre valores analógicos (ADC) e humidade percentual.

Condição do Solo	Valor Analógico (ADC)	Humidade (%)
Solo Seco (Ar)	3900	0%
Limiar Crítico (Murcha)	2900	30%
Volume de Controlo (90ml)	≈ 2640	47%
Limiar Ideal (Paragem)	2300	60%
Solo Saturado (Água)	1750	100%

Um ensaio de validação foi realizado adicionando 90 ml de água a um vaso com substrato seco, resultando numa leitura de aproximadamente 2640 no ADC (47%). Observou-se que, no estado inicial de secura extrema, o solo apresenta um comportamento **hidrofóbico**. Este fenómeno ocorre devido à baixa energia superficial das partículas secas e à presença de ar nos poros, o que dificulta a molhabilidade imediata e exige um maior volume de água para romper a tensão superficial inicial.

Contudo, após a primeira irrigação, o substrato transita para um estado **hidrofilico**. Uma vez que os poros já retêm uma camada de humidade residual, a afinidade química entre as moléculas de água (coesão e adesão) facilita a infiltração. Consequentemente, para elevar a humidade de um limiar intermédio (ex: 30 %) até à capacidade de campo pretendida (60 %), o volume de água necessário será significativamente inferior aos 90 ml iniciais. Este comportamento é característico da **histerese do potencial matricial** do solo, onde a relação entre o conteúdo de água e o potencial de retenção depende do histórico de molhagem e secagem do meio poroso.

3.1.3 Limitações e Fenómenos de Oxidação

É importante salientar que o sensor YL-69, por ser do tipo resistivo, perde credibilidade ao longo do tempo. O processo eletrolítico desencadeado pela passagem de corrente contínua entre os elétrodos em contacto com a terra e a água causa a oxidação progressiva dos seus dentes metálicos. Esta degradação altera a condutividade do componente, exigindo recalibrações periódicas para manter a precisão dos limiares de 30% e 60% definidos para a planta.

3.2 Caracterização Hidráulica da Bomba

Foram realizados quatro ensaios experimentais para determinar o caudal real da bomba peristáltica, cujos resultados constam na Tabela 3.2. A atuação da bomba é gerida através de um relé dos módulos de 4 relés que possui optoacopladores integrados, o que garante a proteção do microcontrolador [4].

Observou-se que a bomba apresenta um comportamento linear (1 ml/s) para tempos de ativação superiores a 30 segundos. No entanto, para o teste de 10 segundos, obteve-se apenas 8 ml. Conclui-se que a bomba necessita de cerca de 1 a 2 segundos iniciais para preencher totalmente a mangueira e estabilizar a pressão de expulsão da água.

Tabela 3.2: Resultados dos ensaios de caudal da bomba.

Tempo (<i>s</i>)	Volume Medido (<i>ml</i>)	Caudal Médio (<i>ml/s</i>)
120	120	1,00
60	59	0,98
30	30	1,00
10	8	0,80

3.3 Dimensionamento Geométrico do Vaso

O vaso possui a geometria de um tronco de cone com raio maior (R) de 6,5 cm, raio menor (r) de 3,75 cm e altura (h) de 10,8 cm. O volume de terra (V) foi calculado pela Equação 3.1:

$$V = \frac{1}{3} \cdot \pi \cdot h \cdot (R^2 + r^2 + R \cdot r) \approx 922,6 \text{ ml} \quad (3.1)$$

Considerando que 1 cm^3 equivale a 1 ml, o volume total de substrato é de aproximadamente 922,6 ml. Este dado permite concluir que um ciclo de rega de 30 segundos (30 ml) representa uma adição hídrica de cerca de 3,25% do volume total do meio de cultura.

3.4 Montagem do Circuito e Diagrama de Ligações

A integração física dos componentes foi realizada centralizando o controlo no ESP32. A Figura 3.2 ilustra o diagrama esquemático das conexões elétricas estabelecidas entre o microcontrolador, os sensores e os atuadores.

O sensor de humidade YL-69 foi alimentado com 3,3 V para garantir a compatibilidade com a lógica do ESP32, sendo o seu pino de sinal analógico conectado a uma porta ADC (GPO 34) configurada apenas para entrada, minimizando interferências.

Para a interface visual, utilizou-se um mostrador LCD com comunicação I2C. A alimentação do display é fornecida pelo pino de 5 V do ESP32, enquanto a comunicação de dados ocorre através das portas padrão I2C: o pino de dados (SDA) ligado ao GPIO 21 e o pino de clock (SCL) ligado ao GPIO 22.

No que respeita à atuação, a bomba peristáltica é gerida através de um módulo de relé. A alimentação lógica do relé (VCC) é partilhada com o LCD, utilizando também a saída de 5 V do ESP32. O circuito de potência da bomba foi isolado da seguinte forma: o terminal Comum (COM) do relé foi conectado à fonte de alimentação externa de 6 V, enquanto o terminal Normalmente Aberto (NO) foi ligado diretamente à bomba. Desta forma, o circuito só fecha (acionando a bomba) quando o ESP32 envia o sinal de ativação, mantendo o sistema seguro e eletricamente isolado.

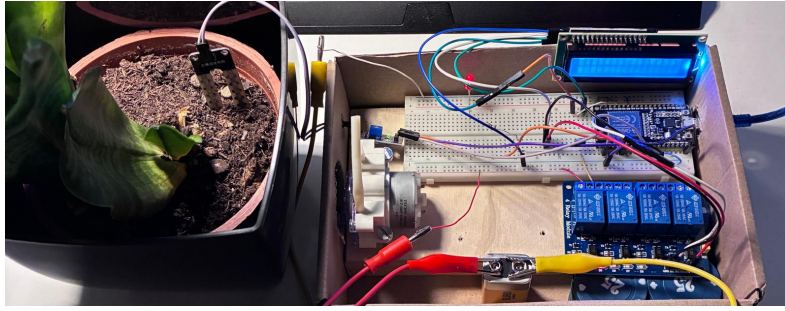


Figura 3.2: Diagrama esquemático das ligações: LCD (I2C), Sensor YL-69 e circuito da bomba via relé.

Capítulo 4

Programação e Firmware

4.1 Lógica de Controle e Necessidades Hídricas

A estratégia de rega foi delineada com base nas necessidades específicas do Lírio-da-paz (*Spathiphyllum*), uma espécie sensível ao stress hídrico. Implementou-se uma lógica de controlo por histerese baseada em três janelas de operação fundamentais:

- **Limiar Crítico (30%):** Ponto de ativação da rega. Abaixo deste valor, o sistema identifica stress hídrico severo e inicia o ciclo de irrigação com uma **Dose Cheia**. Esta consiste numa ativação contínua da bomba durante 30 segundos (aprox. 30 ml), volume necessário para vencer a tensão superficial do solo seco (hidrofobia inicial) e garantir a molhabilidade necessária para evitar a murcha foliar permanente.
- **Zona de Ajuste Fino (50% a 60%):** Intervalo de transição para rega de precisão. Neste estágio, como o solo já recuperou a sua capacidade hidrofílica (absorção rápida), o sistema reduz o tempo de atuação da bomba de 30 para 10 segundos (Micro-Dose). Esta lógica previne a inércia hídrica e o *overshoot*, garantindo uma aproximação suave à meta sem ultrapassar o valor desejado.
- **Limiar de Paragem (60%):** Meta de humidade para o sistema de controlo. Garante que o solo atinge o estado ideal sem causar encharcamento ou asfixia radicular, cessando totalmente a atividade da bomba.
- **Limiar de Planta Saudável (Até 75%):** Define a zona de conforto biológico. Na interface móvel, a planta é exibida como "Saudável" entre os 30% e os 75%. Acima deste valor, o sistema alerta para o risco de apodrecimento das raízes.

Esta janela de funcionamento protege a saúde biológica da planta e evita o desgaste prematuro do relé e da bomba através de acionamentos intermitentes [4].

4.1.1 Implementação do Algoritmo Adaptativo

Para materializar a estratégia descrita, o firmware implementa uma máquina de estados que alterna entre a injeção de água e a espera pela percolação (infiltração). O Excerto 4.1 demonstra como o sistema decide dinamicamente o tempo de ativação da bomba (t_{rega}) com base na leitura instantânea do sensor.

```

1 // Decisao Inteligente do Tempo de Rega
2 long tempoRega = 30000; // Padrao: Dose Cheia (30s)
3
4 // Se o solo ja estiver "quase" bom (>50%), usa apenas ajuste fino
5 if (humidade >= limiarAjuste) {
6     tempoRega = 10000; // Ajuste: Micro-Dose (10s)
7 }
8
9 // Atuacao Fisica
10 digitalWrite(pinoRele, LOW); // Liga Bomba
11 delay(tempoRega);
12 digitalWrite(pinoRele, HIGH); // Desliga Bomba
13
14 // Ciclo de Absorcao (Wait-State)
15 // Aguarda 60s para a agua descer antes de nova leitura critica
16 for(int i = 0; i < 120; i++) {
17     int humInst = lerSensorEstavel();
18     // Se a agua desceu e atingiu a meta, interrompe a espera
19     if (humInst >= humidadeIdeal) break;
20     delay(500);
21 }

```

Listing 4.1: Lógica de decisão de tempo (Dose vs. Micro-Dose) e ciclo de absorção

O ciclo de absorção (passo 3 do código) é fundamental para a estabilidade do sistema. Ao introduzir uma pausa de aproximadamente 60 segundos após cada rega, permite-se que a água infiltre graviticamente até à profundidade do sensor. Isto evita o erro comum de *over-watering*, onde o sistema continuaria a regar excessivamente apenas porque a água ainda não tinha atingido os elétrodos de leitura.

4.2 Comunicação I2C

Para otimizar o uso de memória e garantir total controlo sobre o *timing* do hardware, foi desenvolvido um driver I2C manual para o controlador PCF8574 [5]. O algoritmo segue a lógica de operação de 4-bits, controlando o pino *Enable* para validar a escrita de cada unidade de 4 bit. O LCD requer uma sequência de reset e inicialização específica, conforme datasheet, para garantir que opere em modo 4 bits via PCF8574. Cada byte enviado pelo barramento I2C é seguido de um pulso no pino Enable para assegurar que o comando ou dado seja reconhecido corretamente pelo display.

```

1 #include <wire.h>
2
3 #define SDA_PIN 21
4 #define SCL_PIN 22
5
6 void setup() {
7     Wire.begin(SDA_PIN, SCL_PIN); // Inicializa o barramento I2C
8     lcd_init(); // Sequencia de reset e inicializacao do
9                 // LCD (modo 4 bits via PCF8574)
10 }
11
12 // Envia um byte para o expensor de portas I2C
13 void enviarByteI2C(byte dados) {
14     Wire.beginTransmission(LCD_ADDR);
15     Wire.write(dados | 0x08); // Mantem Backlight ligado (bit 3)
16     Wire.endTransmission();
17 }

```

```

16 }
17
18 // O LCD le os dados quando o pino 'Enable' passa de ALTO para BAIXO
19 void pulsoEnable(byte dados) {
20     enviarByteI2C(dados | 0x04); // Enable HIGH
21     delayMicroseconds(1);
22     enviarByteI2C(dados & ~0x04); // Enable LOW (Momento da leitura)
23     delayMicroseconds(50); // Tempo de processamento do LCD
24 }

```

Listing 4.2: Implementação do Driver I2C e pulso de validação

4.3 Sincronização Temporal e Gestão de Dados Cloud

O sistema utiliza o protocolo NTP (*Network Time Protocol*) para obter a hora exata a partir de servidores globais. Esta sincronização é fundamental para a funcionalidade de **Histórico Semanal**, permitindo que o ESP32 opere de forma autónoma em relação ao calendário [1, 6].

```

1 #include <time.h>
2
3 const char* ntpServer = "pool.ntp.org";
4 const long  gmtOffset_sec = 0; // Portugal (UTC+0)
5 const int   daylightOffset_sec = 0; // 0 no Inverno, 3600 no Verão
6
7 void configurarTempo() {
8     configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
9 }
10
11 int obterDiaSemana() {
12     struct tm timeinfo;
13     if(!getLocalTime(&timeinfo)) return -1;
14     return timeinfo.tm_wday; // Retorna 0 (Dom) a 6 (Sab)
15 }

```

Listing 4.3: Configuração do NTP e obtenção do dia da semana

O firmware utiliza a função `timeinfo.tm_wday` para mapear o caminho de escrita na base de dados (ex: `/historico/dia_1`). Isto permite que a aplicação móvel gere gráficos de barras precisos do estado da planta ao longo dos últimos 7 dias.

4.4 Lógica de Segurança e Tratamento de Dados

Para mitigar o ruído elétrico do sensor YL-69, implementou-se uma filtragem por média móvel. Adicionalmente, o sistema integra uma proteção de "Tanque Vazio" baseada no volume do reservatório (500 ml).

O limite de atuações (N_{limite}) foi determinado assumindo o cenário de consumo máximo, onde todas as regas seriam de "Dose Cheia" ($V_{max} \approx 30$ ml), ignorando a poupança gerada pelas micro-doses. Para prevenir a cavitação da bomba, aplicou-se um fator de segurança (FS) de 20% para volume morto e evaporação:

$$N_{limite} = \frac{V_{tanque} \cdot (1 - FS)}{V_{max}} = \frac{500 \cdot 0,80}{30} \approx 13,3 \quad (4.1)$$

Assim, fixou-se o valor inteiro em **13 regas**. Isto garante que, no pior caso, restam ainda cerca de 110ml no fundo do tanque quando o sistema bloqueia, protegendo a integridade mecânica da bomba peristáltica.

```
1 // Filtragem por media de 10 leituras
2 int lerSensorEstavel() {
3     long soma = 0;
4     for(int i = 0; i < 10; i++){
5         soma += analogRead(pinoSensor);
6         delay(10);
7     }
8     int hum = map(soma/10, valorSeco, valorMolhado, 0, 100);
9     return constrain(hum, 0, 100);
10 }
11
12 // Verificacao de Seguranca do Tanque (Limite: 13 regas)
13 if (contadorRegas >= limiteRegas) {
14     lcd_set_cursor(0, 0);
15     lcd_print("Tanque Vazio! ");
16     lcd_set_cursor(0, 1);
17     lcd_print("Abastecer Agua ");
18
19     if (Firebase.ready()) {
20         Firebase.RTDB.setString(&fbdo, "/rega/ultima", "ERRO: Tanque
Vazio");
21
22         // Logica de Reset Remoto via App (se utilizador enviou 0)
23         if (Firebase.RTDB.getInt(&fbdo, "/status/numero_regas")) {
24             if (fbdo.intData() == 0) {
25                 contadorRegas = 0;
26                 lcd_command(0x01); // Limpa o ecrã ao reiniciar
27                 return;
28             }
29         }
30     }
31     delay(5000);
32     return;
33 }
```

Listing 4.4: Algoritmo de filtragem analógica e segurança do reservatório

Capítulo 5

Aplicação Móvel e Cloud

5.1 Desenvolvimento da Aplicação Móvel

Inicialmente, seguindo o conselho do Prof. Manuel Raposo, o desenvolvimento da aplicação móvel foi iniciado utilizando a plataforma **MIT App Inventor**. Esta ferramenta foi a primeira escolha para prototipagem rápida da interface de controlo.

No entanto, durante os testes preliminares, verificaram-se instabilidades recorrentes na conexão com os servidores na Europa, o que comprometia a latência e a usabilidade do projeto em tempo real. Como alternativa técnica, migrou-se o desenvolvimento integralmente para a plataforma **Thunkable** [7], que demonstrou maior robustez e estabilidade na integração com a Firebase. Complementarmente, a criação dos elementos gráficos e a identidade visual da interface foram auxiliadas por ferramentas de inteligência artificial [8].

5.2 Google Firebase (Realtime Database)

A infraestrutura de dados do SmartPot assenta na *Realtime Database* da Google Firebase. Esta base de dados NoSQL permite a sincronização bidirecional de dados entre o ESP32 e a aplicação móvel com latência reduzida [1].

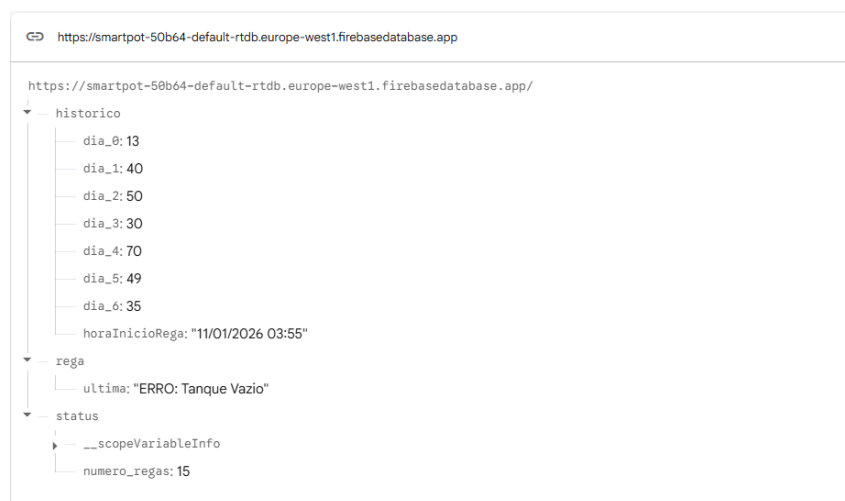


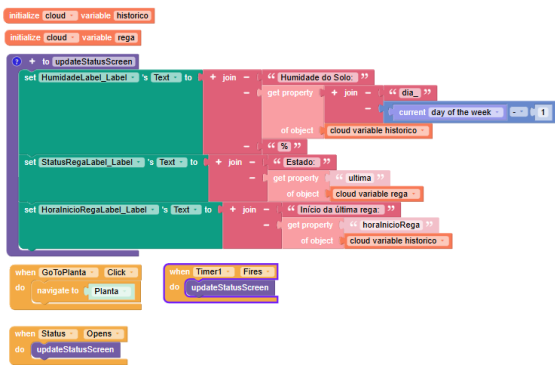
Figura 5.1: Estrutura de dados JSON na consola do Firebase: nós de histórico e status.

Como ilustrado na Figura 5.1, os dados são organizados em caminhos específicos:

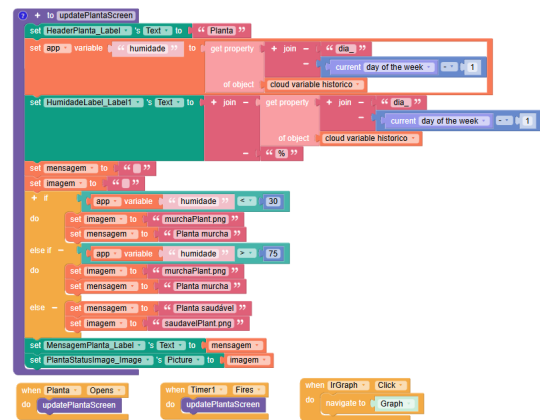
- `/historico`: Armazena as médias de humidade diárias enviadas pelo ESP32 via NTP.
- `/status`: Contém variáveis de controlo, como o contador de regas para segurança do tanque.

5.3 Lógica de Programação (Thunkable)

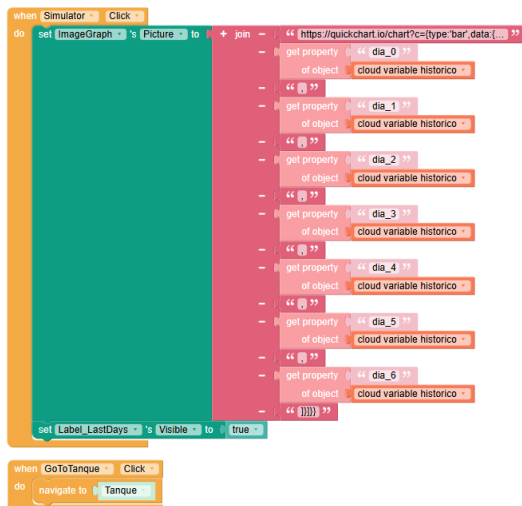
A inteligência da interface móvel foi desenvolvida na plataforma Thunkable através de programação visual por blocos. Esta abordagem permite gerir eventos complexos de forma modular.



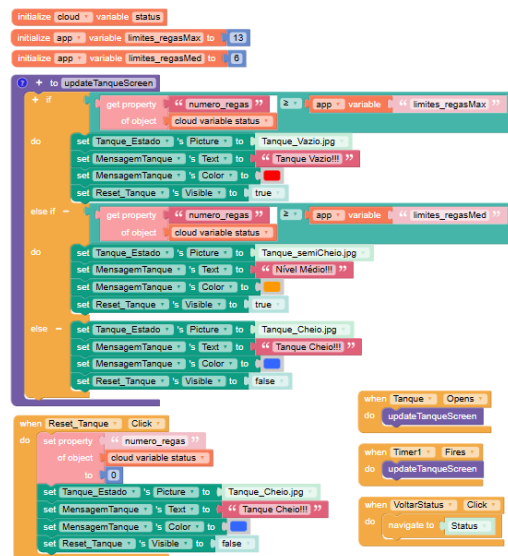
(a) Status: Monitorização em Tempo Real.



(b) Planta: Interface Visual Adaptativa.



(c) Gráfico: Histórico Semanal.



(d) Segurança: Gestão e Alerta de Tanque.

Figura 5.2: Lógica de blocos da App SmartPot.

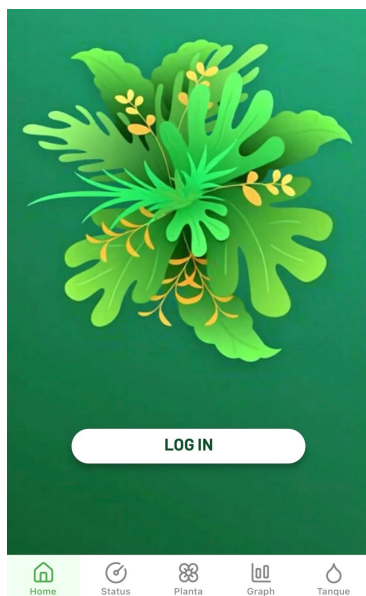
5.4 Análise da Lógica de Programação (Thunkable)

A inteligência da interface móvel foi estruturada de forma modular, permitindo a sincronização bidirecional com a *Realtime Database* do Firebase através de eventos temporizados e lógicas condicionais. Abaixo descreve-se o funcionamento técnico de cada bloco apresentado na Figura 5.2:

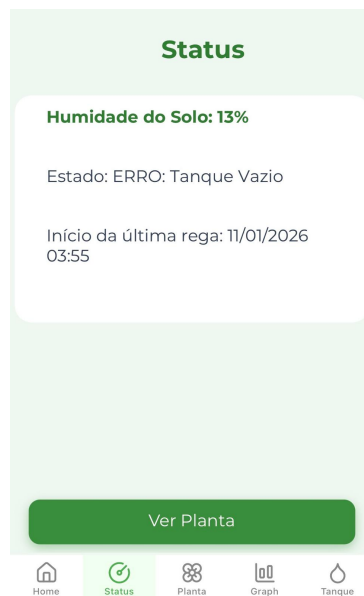
- a) **Status – Sincronização Dinâmica e Monitorização:** Esta alínea foca-se na automação da leitura de dados. Utiliza um componente *Timer* que dispara uma função de atualização a cada 3 segundos. Para garantir que a aplicação apresenta o valor correto sem intervenção manual, implementou-se um algoritmo que obtém o dia da semana do sistema operativo e subtrai 1 unidade, criando dinamicamente o caminho de busca (ex: "dia_1") através de um bloco *join*. Isto permite que a *label* de humidade reflita sempre os dados guardados na pasta correspondente ao dia atual no Firebase.
- b) **Planta – Interface Visual Adaptativa:** A lógica deste ecrã traduz dados numéricos em informação visual intuitiva. Utiliza uma estrutura *if-else* que compara a humidade atual com os limiares biológicos definidos: se o valor for inferior a 30% ou superior a 75%, a aplicação altera a propriedade *Picture* para uma imagem de planta murcha e atualiza o texto de aviso. Caso o valor esteja no intervalo ideal, a interface valida o estado como "Saudável", facilitando o diagnóstico rápido pelo utilizador.
- c) **Gráfico – Compilação de Dados e Integração API:** Este bloco gere a visualização do histórico semanal através da integração com a API externa *QuickChart*. O algoritmo utiliza um bloco *join* extenso para concatenar a URL base da API com os valores de humidade de todos os sete dias da semana (*dia_0* a *dia_6*) extraídos da *cloud variable historico*. O resultado é um URL dinâmico que, ao ser carregado num componente de imagem, renderiza automaticamente um gráfico de barras atualizado.
- d) **Segurança – Gestão e Alerta de Tanque:** Esta secção controla a integridade do hardware através da monitorização da variável *numero_regas*. A lógica altera a interface conforme três estados de alerta: "Tanque Cheio", "Nível Médio" e "Tanque Vazio" (ativado quando o contador atinge ≥ 13 regas). Para permitir a continuidade do sistema, foi implementado um evento de clique no botão *Reset_Tanque* que reatribui o valor 0 à variável no Firebase, desbloqueando o funcionamento da bomba após o reabastecimento físico.

5.5 Interface e Fluxo do Utilizador

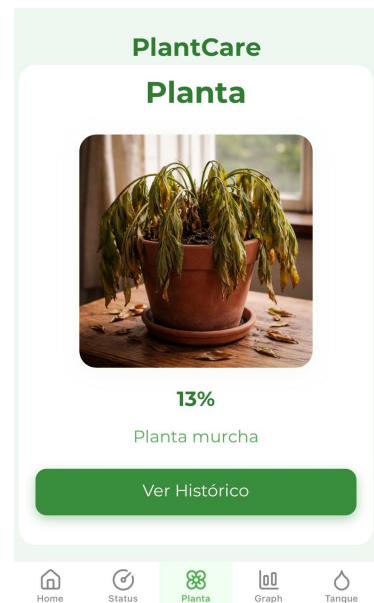
A interface final foi projetada para oferecer uma experiência intuitiva. A Figura 5.3 apresenta os ecrãs principais por ordem de utilização, desde a autenticação até à gestão de erros críticos.



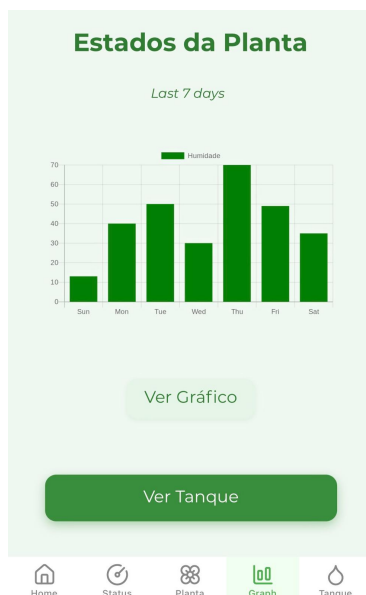
(a) Ecrã de Login



(b) Status do Sistema



(c) Feedback Visual



(d) Histórico Semanal



(e) Gestão de Erros

Figura 5.3: Fluxo de navegação e funcionalidades da aplicação móvel.

Capítulo 6

Análise Económica

O investimento total foi de 22,00 (€).

Tabela 6.1: Orçamento detalhado.

Componente	Custo (€)
ESP32	6,00
Bomba e Relé	8,00
Sensores e LCD	6,00
Outros	2,00
TOTAL	22,00

O valor total de 22,00 (€) reflete uma solução de baixo custo, especialmente quando comparada com sistemas de rega inteligente comerciais disponíveis no mercado. Esta viabilidade económica é possível graças à utilização de componentes de *hardware open-source* e ao aproveitamento de plataformas *cloud* gratuitas para estudantes. O investimento foca-se nos elementos ativos (ESP32 e Bomba), enquanto a inteligência do sistema foi integrada via software, eliminando a necessidade de controladores industriais proprietários de elevado custo.

Capítulo 7

Conclusão

7.1 Síntese do Trabalho e Cumprimento dos Objetivos

O desenvolvimento do projeto **SmartPot** realizado possibilitou a consolidação dos conhecimentos obtidos no âmbito da cadeira de Tecnologias de Acionamento e Comando, bem como a aquisição de conhecimentos que transcendem o proposto e esperado em ambiente de aula. Permitiu cumprir com sucesso todos os objetivos inicialmente propostos. Foi possível projetar e implementar um sistema funcional capaz de:

1. **Automatizar a Rega:** O algoritmo de histerese demonstrou eficácia na manutenção da humidade do solo dentro dos parâmetros vitais do Lírio-da-paz, evitando a intervenção humana diária.
2. **Monitorização Remota:** A integração entre o ESP32 e a Google Firebase, aliada à aplicação móvel desenvolvida no Thunkable, garantiu a visualização de dados em tempo real e o acesso ao histórico semanal com baixa latência.
3. **Robustez e Segurança:** A implementação de mecanismos de proteção, como a deteção lógica de "Tanque Vazio" e o tratamento de dados do sensor por média móvel, conferiu ao protótipo um nível de fiabilidade superior a soluções de rega temporizada simples.

Conclui-se que a arquitetura proposta (Sensor → ESP32 → Cloud → App) é uma solução viável, de baixo custo (aprox. 22,00€) e escalável para aplicações de agricultura doméstica.

7.2 Perspetivas de Evolução e Trabalhos Futuros

Apesar dos resultados satisfatórios, identificaram-se áreas onde a introdução de novos componentes e lógicas de controlo elevaria o sistema para um patamar semiprofissional:

- **Sensor de Nível Ultrassónico (HC-SR04):** Atualmente, a deteção de falta de água é feita por software (dedução após 15 falhas de rega). A instalação de um sensor ultrassónico no topo do reservatório permitiria medir o volume real de água em litros. Isto eliminaria o risco de a bomba trabalhar em seco durante os ciclos de deteção de erro e permitiria enviar notificações preventivas de "Nível Baixo" antes de a água acabar.

- **Implementação de Controlo PID:** O sistema atual opera numa lógica *ON/OFF*. Para plantas muito sensíveis, sugere-se a evolução para um controlo Proporcional-Integral-Derivativo (PID). Através da modulação PWM (*Pulse Width Modulation*) na bomba, seria possível injetar água com caudal variável proporcionalmente ao erro de humidade, evitando o *overshoot* (picos de humidade acima do desejado) e garantindo uma estabilização suave no *setpoint*.
- **Integração de Sensores Ambientais:** Para uma análise biológica mais completa, propõe-se a adição de:
 - **Sensor de Temperatura e Humidade do Ar (DHT22):** Para correlacionar a evaporação da água do solo com o calor ambiente.
 - **Sensor de Luminosidade (LDR):** Para garantir que a rega ocorre preferencialmente em horários de menor insolação ou para monitorizar se a planta está a receber luz suficiente para a fotossíntese.
- **Autonomia Energética:** Implementação de modos de *Deep Sleep* no ESP32 e adição de um painel solar com carregador de baterias de Lítio (TP4056), tornando o vaso independente da rede elétrica.

Bibliografia

- [1] Mobizt. Firebase esp client library documentation. GitHub Repository, 2024. <https://github.com/mobizt/Firebase-ESP-Client>.
- [2] Espressif Systems. *ESP-WROOM-32 Datasheet*, 2023. Disponível em: <https://www.espressif.com>.
- [3] YouTube Channel. Sensor de humidade (yl-69), 2024. Vídeo: <https://www.youtube.com/watch?v=QPH9aXS2bio>.
- [4] SunFounder. *TS0011 Datasheet: 4 Channel 5V Relay Module*. SunFounder, n.d. Disponível em: <https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/5773/TS0011%20DATASHEET.pdf>.
- [5] Crystalfontz. *LCD 16x2 Hitachi HD44780 Datasheet*, 2022. Disponível em: <https://www.alldatasheet.com>.
- [6] YouTube Channel. Esp32 data e hora(npt) tutorial, 2024. Vídeo: <https://www.youtube.com/watch?v=wmxptpd6aWc>.
- [7] YouTube Channel. App thunkable, 2024. Vídeo: <https://www.youtube.com/@Thunkable>.
- [8] OpenAI. Chatgpt: Optimizing language models for dialogue. Large Language Model, 2024. Versão GPT-4. Utilizado para geração de ativos gráficos da aplicação e apoio na estruturação do projeto. Disponível em: <https://chat.openai.com>.
- [9] YouTube Channel. Esp32 firebase tutorial, 2024. Vídeo: <https://www.youtube.com/watch?v=a092B-K4TnQ>.